

Contents

Building a CRUD App	2
Step 1: XAMPP MySQL Database Setup	2
Step 2: Create the Backend API (Node.js/Express)	2
Step 3: React Frontend Dependencies & Tailwind Setup	3
Step 4: Zod Schema & Types	4
Step 5: Redux Toolkit Setup.....	4
Step 6: UI Components	6
Step 7: Final Integration	11

Building a CRUD App

Step 1: XAMPP MySQL Database Setup

1. Open the XAMPP Control Panel and start the **MySQL** module.
2. Open your browser and navigate to <http://localhost/phpmyadmin>.
3. Create a new database named `sampledb`.
4. Go to the **SQL** tab and run the following query to create your table:

```
CREATE TABLE products (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  name VARCHAR(255) NOT NULL,  
  description TEXT,  
  unitprice DECIMAL(10, 2) NOT NULL,  
  stocks INT NOT NULL  
);
```

Step 2: Create the Backend API (Node.js/Express)

We need a simple API to handle Axios requests from your React app and execute SQL queries.

1. Open a new terminal outside your React project folder and initialize a backend folder:

```
mkdir backend && cd backend  
npm init -y  
npm install express mysql2 cors dotenv  
npm install --save-dev nodemon
```

2. Create a file named `server.js` and paste the following code:

```
const express = require('express');  
const mysql = require('mysql2');  
const cors = require('cors');  
  
const app = express();  
app.use(cors());  
app.use(express.json());  
  
const db = mysql.createPool({  
  host: 'localhost',  
  user: 'root',    // Default XAMPP user  
  password: '',    // Default XAMPP password is empty  
  database: 'sampledb'  
});  
  
// Get all products  
app.get('/api/products', (req, res) => {  
  db.query('SELECT * FROM products', (err, results) => {  
    if (err) return res.status(500).json(err);  
    res.json(results);  
  });  
});
```

```
// Add a product
app.post('/api/products', (req, res) => {
  const { name, description, unitprice, stocks } = req.body;
  const sql = 'INSERT INTO products (name, description, unitprice, stocks) VALUES (?, ?, ?, ?)';
  db.query(sql, [name, description, unitprice, stocks], (err, result) => {
    if (err) return res.status(500).json(err);
    res.json({ id: result.insertId, ...req.body });
  });
});

// Update a product
app.put('/api/products/:id', (req, res) => {
  const { name, description, unitprice, stocks } = req.body;
  const sql = 'UPDATE products SET name=?, description=?, unitprice=?, stocks=? WHERE id=?';
  db.query(sql, [name, description, unitprice, stocks, req.params.id], (err) => {
    if (err) return res.status(500).json(err);
    res.json({ id: Number(req.params.id), ...req.body });
  });
});

// Delete a product
app.delete('/api/products/:id', (req, res) => {
  db.query('DELETE FROM products WHERE id=?', [req.params.id], (err) => {
    if (err) return res.status(500).json(err);
    res.json({ message: 'Deleted successfully' });
  });
});

app.listen(5000, () => console.log('Server running on port 5000'));
```

3. Run the backend server using `npx nodemon server.js`.

Step 3: React Frontend Dependencies & Tailwind Setup

In your Vite React TS project terminal, install the necessary packages:

```
npm install @reduxjs/toolkit react-redux axios zod
```

Install TailwindCSS

```
npm install -D tailwindcss @tailwindcss/vite
```

Update vite.config.ts

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'
import tailwindcss from '@tailwindcss/vite'

export default defineConfig({
  plugins: [
```

```
    react(),
    tailwindcss(),
  ],
})
```

Create/Update file *tailwind.config.js* (at the root of your project)

```
/** @type {import('tailwindcss').Config} */
export default {
  content: [
    './index.html',
    './src/**/*.{ts,tsx,js,jsx}',
  ],
  theme: {
    extend: {},
  },
  plugins: [],
}
```

Update existing file *src/index.css* (Clear the file completely and add these 3 lines):

```
Import "tailwindcss";
```

Step 4: Zod Schema & Types

Create *src/types.ts* to share interfaces and the Zod schema across your app.

```
import { z } from 'zod';

export const productSchema = z.object({
  name: z.string().min(2, "Name must be at least 2 characters"),
  description: z.string().min(5, "Description is too short"),
  unitprice: z.coerce.number().positive("Price must be greater than 0"),
  stocks: z.coerce.number().int().nonnegative("Stocks cannot be negative"),
});

export type ProductInput = z.infer<typeof productSchema>;

export interface Product extends ProductInput {
  id: number;
}
```

Step 5: Redux Toolkit Setup

1. Create *src/store/productSlice.ts*:

```
import { createSlice, createAsyncThunk, PayloadAction } from '@reduxjs/toolkit';
import axios from 'axios';
import { Product, ProductInput } from '../types';

const API_URL = 'http://localhost:5000/api/products';
```

```

interface ProductState {
  items: Product[];
  status: 'idle' | 'loading' | 'succeeded' | 'failed';
  error: string | null;
  searchQuery: string;
  sortConfig: { key: keyof Product; direction: 'asc' | 'desc' } | null;
}

const initialState: ProductState = {
  items: [],
  status: 'idle',
  error: null,
  searchQuery: '',
  sortConfig: null,
};

export const fetchProducts = createAsyncThunk('products/fetch', async () => {
  const response = await axios.get(API_URL);
  return response.data;
});

export const addProduct = createAsyncThunk('products/add', async (product: ProductInput) => {
  const response = await axios.post(API_URL, product);
  return response.data;
});

export const updateProduct = createAsyncThunk('products/update', async ({ id, data }: { id: number; data: ProductInput }) => {
  const response = await axios.put(`${API_URL}/${id}`, data);
  return response.data;
});

export const deleteProduct = createAsyncThunk('products/delete', async (id: number) => {
  await axios.delete(`${API_URL}/${id}`);
  return id;
});

const productSlice = createSlice({
  name: 'products',
  initialState,
  reducers: {
    setSearchQuery: (state, action: PayloadAction<string>) => {
      state.searchQuery = action.payload;
    },
    setSortConfig: (state, action: PayloadAction<{ key: keyof Product; direction: 'asc' | 'desc' }>) => {
      state.sortConfig = action.payload;
    },
  },
});

```

```

extraReducers: (builder) => {
  builder
    .addCase(fetchProducts.pending, (state) => { state.status = 'loading'; })
    .addCase(fetchProducts.fulfilled, (state, action) => {
      state.status = 'succeeded';
      state.items = action.payload;
    })
    .addCase(fetchProducts.rejected, (state, action) => {
      state.status = 'failed';
      state.error = action.error.message || 'Failed to fetch products';
    })
    .addCase(addProduct.fulfilled, (state, action) => { state.items.push(action.payload); })
    .addCase(updateProduct.fulfilled, (state, action) => {
      const index = state.items.findIndex(p => p.id === action.payload.id);
      if (index !== -1) state.items[index] = action.payload;
    })
    .addCase(deleteProduct.fulfilled, (state, action) => {
      state.items = state.items.filter(p => p.id !== action.payload);
    });
},
});

export const { setSearchQuery, setSortConfig } = productSlice.actions;
export default productSlice.reducer;

```

2. Create src/store/index.ts:

```

import { configureStore } from '@reduxjs/toolkit';
import productReducer from './productSlice';

export const store = configureStore({
  reducer: {
    products: productReducer,
  },
});

export type RootState = ReturnType<typeof store.getState>;
export type AppDispatch = typeof store.dispatch;

```

Step 6: UI Components

1. The Confirmation Dialog (src/components/ConfirmDialog.tsx)

A reusable modal for edit and delete actions.

```

import React from 'react';

interface Props {
  isOpen: boolean;
  title: string;
  message: string;
}

```

```

onConfirm: () => void;
onCancel: () => void;
}

export const ConfirmDialog: React.FC<Props> = ({ isOpen, title, message, onConfirm, onCancel }) => {
  if (!isOpen) return null;
  return (
    <div className="fixed inset-0 bg-black bg-opacity-50 flex justify-center items-center p-4">
      <div className="bg-white rounded p-6 shadow-lg max-w-sm w-full">
        <h3 className="text-lg font-bold mb-2">{title}</h3>
        <p className="text-gray-600 mb-4">{message}</p>
        <div className="flex justify-end gap-2">
          <button onClick={onCancel} className="px-4 py-2 bg-gray-200 rounded hover:bg-gray-300">Cancel</button>
          <button onClick={onConfirm} className="px-4 py-2 bg-blue-600 text-white rounded hover:bg-blue-700">Confirm</button>
        </div>
      </div>
    </div>
  );
};

```

2. Main App Component (src/App.tsx)

This file combines Redux state, Zod validation, Tailwind styling, and CRUD logic into one cohesive view.

```

import React, { useEffect, useState } from 'react';
import { useDispatch, useSelector } from 'react-redux';
import { AppDispatch, RootState } from './store';
import { fetchProducts, addProduct, updateProduct, deleteProduct, setSearchQuery, setSortConfig } from './store/productSlice';
import { productSchema, ProductInput, Product } from './types';
import { z } from 'zod';
import { ConfirmDialog } from './components/ConfirmDialog';

function App() {
  const dispatch = useDispatch<AppDispatch>();
  const { items, status, error, searchQuery, sortConfig } = useSelector((state: RootState) => state.products);

  const [formData, setFormData] = useState<ProductInput>({ name: '', description: '', unitprice: 0, stocks: 0 });
  const [formErrors, setFormErrors] = useState<Record<string, string>>({});
  const [editingId, setEditingId] = useState<number | null>(null);
  const [successMsg, setSuccessMsg] = useState('');

  // Dialog States
  const [dialogConfig, setDialogConfig] = useState<{isOpen: boolean, action: 'delete'|'edit'|null, payload?: any}>({ isOpen: false, action: null });

```

```

useEffect(() => {
  if (status === 'idle') {
    dispatch(fetchProducts());
  }
}, [status, dispatch]);

// Derived State: Search & Sort
const filteredAndSortedItems = [...items]
  .filter(item => item.name.toLowerCase().includes(searchQuery.toLowerCase()) ||
    item.description.toLowerCase().includes(searchQuery.toLowerCase()))
  .sort((a, b) => {
    if (!sortConfig) return 0;
    if (a[sortConfig.key] < b[sortConfig.key]) return sortConfig.direction === 'asc' ? -1 : 1;
    if (a[sortConfig.key] > b[sortConfig.key]) return sortConfig.direction === 'asc' ? 1 : -1;
    return 0;
  });

const handleSort = (key: keyof Product) => {
  let direction: 'asc' | 'desc' = 'asc';
  if (sortConfig && sortConfig.key === key && sortConfig.direction === 'asc') direction = 'desc';
  dispatch(setSortConfig({ key, direction }));
};

const handleInputChange = (e: React.ChangeEvent<HTMLInputElement | HTMLTextAreaElement>)
=> {
  setFormData({ ...formData, [e.target.name]: e.target.value });
};

const showSuccess = (msg: string) => {
  setSuccessMsg(msg);
  setTimeout(() => setSuccessMsg(""), 3000);
};

const handleSubmit = (e: React.FormEvent) => {
  e.preventDefault();
  try {
    productSchema.parse(formData); // Zod Validation
    setFormErrors({});

    if (editingId) {
      setDialogConfig({ isOpen: true, action: 'edit', payload: { id: editingId, data: formData } });
    } else {
      dispatch(addProduct(formData));
      showSuccess('Product added successfully!');
      setFormData({ name: "", description: "", unitprice: 0, stocks: 0 });
    }
  } catch (err) {
    if (err instanceof z.ZodError) {
      const errors: Record<string, string> = {};

```

```

    err.errors.forEach(e => { if (e.path[0]) errors[e.path[0].toString()] = e.message; });
    setFormErrors(errors);
  }
}
};

const executeAction = () => {
  if (dialogConfig.action === 'delete') {
    dispatch(deleteProduct(dialogConfig.payload));
    showSuccess('Product deleted successfully!');
  } else if (dialogConfig.action === 'edit') {
    dispatch(updateProduct(dialogConfig.payload));
    showSuccess('Product updated successfully!');
    setEditingId(null);
    setFormData({ name: '', description: '', unitprice: 0, stocks: 0 });
  }
  setDialogConfig({ isOpen: false, action: null });
};

return (
  <div className="min-h-screen bg-gray-50 p-8">
    <div className="max-w-5xl mx-auto space-y-6">
      <h1 className="text-3xl font-bold text-gray-800">Product Management</h1>

      {/* Success Alert */}
      {successMsg && <div className="p-4 bg-green-100 text-green-700 rounded shadow">{successMsg}</div>}

      {/* Form Section */}
      <div className="bg-white p-6 rounded shadow">
        <h2 className="text-xl font-semibold mb-4">{editingId ? 'Edit Product' : 'Add New Product'}</h2>
        <form onSubmit={handleSubmit} className="grid grid-cols-2 gap-4">
          <div>
            <input name="name" value={formData.name} onChange={handleInputChange}
placeholder="Name" className="w-full border p-2 rounded" />
            {formErrors.name && <p className="text-red-500 text-sm mt-1">{formErrors.name}</p>}
          </div>
          <div>
            <input name="unitprice" type="number" step="0.01" value={formData.unitprice || ''}
onChange={handleInputChange} placeholder="Price" className="w-full border p-2 rounded" />
            {formErrors.unitprice && <p className="text-red-500 text-sm mt-1">{formErrors.unitprice}</p>}
          </div>
          <div>
            <input name="stocks" type="number" value={formData.stocks || ''}
onChange={handleInputChange} placeholder="Stocks" className="w-full border p-2 rounded" />
            {formErrors.stocks && <p className="text-red-500 text-sm mt-1">{formErrors.stocks}</p>}
          </div>
        </form>
      </div>
    </div>
  </div>
);

```

```

    <div className="col-span-2">
      <textarea name="description" value={formData.description} onChange={handleInputChange}
placeholder="Description" className="w-full border p-2 rounded" />
      {formErrors.description && <p className="text-red-500 text-sm mt-
1">{formErrors.description}</p>}
    </div>
    <div className="col-span-2 flex gap-2">
      <button type="submit" className="bg-blue-600 text-white px-4 py-2 rounded hover:bg-blue-
700">
        {editingId ? 'Update Product' : 'Save Product'}
      </button>
      {editingId && (
        <button type="button" onClick={() => { setEditingId(null); setFormData({ name: '', description:
'', unitprice: 0, stocks: 0 }); setFormErrors({}); }} className="bg-gray-400 text-white px-4 py-2 rounded
hover:bg-gray-500">
          Cancel
        </button>
      )}
    </div>
  </form>
</div>

{/* Table & Tools Section */}
<div className="bg-white p-6 rounded shadow">
  <input type="text" placeholder="Search by name or description..." onChange={(e) =>
dispatch(setSearchQuery(e.target.value))} className="w-full border p-2 rounded mb-4" />

  {status === 'loading' && <p className="text-gray-500">Loading data...</p>}
  {status === 'failed' && <p className="text-red-500">{error}</p>}
  {status === 'succeeded' && filteredAndSortedItems.length === 0 && <p className="text-gray-
500">No products found.</p>}

  {status === 'succeeded' && filteredAndSortedItems.length > 0 && (
    <div className="overflow-x-auto">
      <table className="w-full text-left border-collapse">
        <thead>
          <tr className="bg-gray-100 border-b">
            <th className="p-3 cursor-pointer hover:bg-gray-200" onClick={() =>
handleSort('name')}>Name {sortConfig?.key === 'name' ? (sortConfig.direction === 'asc' ? '↑' : '↓') :
''}</th>
            <th className="p-3">Description</th>
            <th className="p-3 cursor-pointer hover:bg-gray-200" onClick={() =>
handleSort('unitprice')}>Price {sortConfig?.key === 'unitprice' ? (sortConfig.direction === 'asc' ? '↑' :
'↓') : ''}</th>
            <th className="p-3 cursor-pointer hover:bg-gray-200" onClick={() =>
handleSort('stocks')}>Stocks {sortConfig?.key === 'stocks' ? (sortConfig.direction === 'asc' ? '↑' : '↓') :
''}</th>
            <th className="p-3 text-right">Actions</th>
          </tr>

```

```

    </thead>
    <tbody>
      {filteredAndSortedItems.map((p) => (
        <tr key={p.id} className="border-b hover:bg-gray-50">
          <td className="p-3 font-medium">{p.name}</td>
          <td className="p-3 text-sm text-gray-600">{p.description}</td>
          <td className="p-3">${Number(p.unitprice).toFixed(2)}</td>
          <td className="p-3">{p.stocks}</td>
          <td className="p-3 text-right space-x-2">
            <button onClick={() => { setEditingId(p.id); setFormData(p); setFormErrors({}); }}
className="text-blue-600 hover:underline">Edit</button>
            <button onClick={() => setDialogConfig({ isOpen: true, action: 'delete', payload: p.id })}
className="text-red-600 hover:underline">Delete</button>
          </td>
        </tr>
      )})
    </tbody>
  </table>
</div>
)}
</div>
</div>

<ConfirmDialog
  isOpen={dialogConfig.isOpen}
  title={dialogConfig.action === 'delete' ? 'Delete Product' : 'Confirm Edit'}
  message={`Are you sure you want to ${dialogConfig.action} this item?`}
  onConfirm={executeAction}
  onCancel={() => setDialogConfig({ isOpen: false, action: null })}
/>
</div>
);
}

export default App;

```

Step 7: Final Integration

Wrap your app with the Redux Provider in src/main.tsx:

```

import React from 'react'
import ReactDOM from 'react-dom/client'
import App from './App.tsx'
import './index.css'
import { Provider } from 'react-redux'
import { store } from './store'

ReactDOM.createRoot(document.getElementById('root')!).render(
  <React.StrictMode>

```

```
<Provider store={store}>  
  <App />  
</Provider>  
</React.StrictMode>  
)
```